

A Low-Latency Hardware-Accelerated Gesture-to-Speech System for Wearable Assistive Applications

Abstract

Assistive communication systems for speech-impaired users require low latency, hardware determinism, and energy-efficient edge execution. This paper presents an FPGA-accelerated wireless Gesture-to-Speech system that synthesizes a compact 11-bit binary gesture code from combined flex sensors and accelerometer data, transmitted via Bluetooth to an RTL hardware decoder. A hardware-aware fixed-point quantization strategy is developed to minimize datapath width while preserving gesture separability. The decoder is implemented in combinational logic to achieve deterministic timing on an Intel MAX 10 FPGA. Comprehensive analyses of latency, throughput, resource utilization, and power behaviour are provided. An ASIC migration framework with standard-cell mapping, clock tree synthesis, and leakage optimisation is also outlined. Experimental results demonstrate nanosecond-scale decode latency, minimal FPGA resources, and significant improvements in predictability relative to microcontroller and CNN-accelerated FPGA approaches. This work provides a scalable template for low-power, real-time assistive interfaces with deep hardware–software co-design.

Keywords

Fixed-point quantization, wireless sensor fusion, FPGA combinational decoder, ASIC migration, deterministic latency, hand gesture recognition

I. Introduction

A. Motivation

Hand gesture recognition has evolved from computer vision approaches to wearable sensor fusion due to the need for lightweight, privacy-preserving assistive interfaces for speech-impaired users [1], [2]. Systems intended for medical-assistive use must achieve predictable timing, low power consumption, and robust edge inference [3]. While microcontroller-based systems

are low cost, they suffer from non-deterministic polling loops and firmware overhead that undermine real-time performance [4]. Conversely, computationally heavy approaches such as convolutional neural networks (CNNs) can achieve high classification accuracy but consume significant memory and DSP resources, reducing suitability for compact edge applications [5].

B. Technical Challenges

Key challenges in assistive gesture platforms include:

1. Real-time determinism: Predictable timing at the hardware level.
2. Resource efficiency: Low logic area and power without large BRAM/DSP footprints.
3. Scalability: Support for an extended gesture vocabulary with minimal hardware complexity.

C. Contributions

This paper introduces a binary gesture encoding strategy, a hardware-driven decoder implemented in pure combinational logic, analytic modelling of fixed-point quantization, and an ASIC migration methodology. The approach contrasts with neural accelerators by leveraging Boolean decoding for constant-time decision logic.

II. Related Work and Research Gap

A. MCU and Hybrid Systems

Early assistive gesture systems used microcontrollers (e.g., Arduino) for interpreting flex and inertial data and generating audio feedback [4], [6]. These systems are cost-effective but exhibit varying latency due to firmware scheduling and lack deterministic hardware support.

B. FPGA-Based CNN Acceleration

Research has explored FPGA accelerators for gesture recognition using quantized CNNs to reduce inference complexity:

1. A QDS-CNN system implemented on an FPGA achieved microsecond-scale inference with 1-bit/4-bit quantization [7].
2. CNN accelerators on Zynq SoC demonstrated improvements via depthwise separable convolutions [8].
3. Lightweight CNN architectures on PYNQ-Z2 achieved real-time classification but required significant DSP and BRAM resources [9].

While effective, these methods introduce **pipelined latency variance and memory overhead**, which limit suitability for tightly constrained assistive hardware.

C. Boolean and Rule-Based FPGA Designs

Limited work exists on pure boolean logic implementations for gesture decoding. Rule-based systems offer constant latency and low hardware cost but have seldom been applied to fused sensor inputs or wireless architectures.

Research Gap: Existing FPGA gesture systems either depend heavily on neural accelerators with high resource use or microcontrollers with unpredictable timing. A combinational LUT-based decoding method that leverages sensor fusion and fixed-point modelling has not been comprehensively studied.

III. System Architecture Overview

A. Wearable Sensor Subsystem

The gesture input hardware consists of:

1. Five flex sensors for finger bending.
2. An MPU6050 inertial measurement unit (IMU) measuring 3-axis acceleration.
3. An Arduino Nano performing ADC acquisition and thresholding.
4. HC-05 Bluetooth module operating at 9600–115200 baud in master mode.

The sensors are sampled at rates sufficient to capture human motion bandwidth (<100 Hz), and data are encoded into an 11-bit binary vector.

B. 11-Bit Binary Gesture Encoding

The gesture vector is defined as:

$$G = \{F_1, F_2, F_3, F_4, F_5, X_{1:0}, Y_{1:0}, Z_{1:0}\}$$

where,

1. F_i are binary flex states.
2. $X_{1:0}, Y_{1:0}, Z_{1:0}$ represent 2-bit quantized acceleration for each axis.

This vector supports up to $2^{11}=20482$ unique codes.

C. Wireless Communication Layer

Bluetooth serial transmission encapsulates the 11-bit data in 13-bit UART frames (start/data/stop bits). The FPGA receives the packet through a UART receiver module and buffers it for decoding.

D. FPGA Decoding Subsystem

On the receiver side, an Arduino Nano interfaces with the HC-05 and forwards decoded bytes to the FPGA. The FPGA implements:

1. UART deserializer
2. Input register for handshake synchronization
3. Gesture decoder — fully combinational logic
4. Audio trigger controller driving DFPlayer Mini

This pipeline minimizes sequential logic and enables constant delay between gesture arrival and audio command issuance.

IV. Mathematical Modelling

A. Flex Sensor Modelling

Flex sensors exhibit resistance variation:

$R_f(\theta) = R_0 + k$, where R_0 is the base resistance and k is the sensitivity coefficient.

A voltage divider converts resistance to an analog voltage:

$$V_{out} = \frac{V_{cc} R_f}{R_f + R_{ref}}$$

Threshold selection is guided by calibration across bending ranges to maximize gesture class

separability. A fixed threshold V_{th} distinguishes binary states:

$$F_i = \begin{cases} 1 & V_{out} > V_{th} \\ 0 & V_{out} \leq V_{th} \end{cases}$$

B. Accelerometer Quantization

Acceleration values a_x , a_y , a_z are quantized, which yields a 2-bit representation per axis reflecting coarse motion direction. The bin boundaries were chosen empirically to capture dominant orientations while minimizing overlap.

C. Fixed-Point Arithmetic Trade-offs

Floating-point units are costly in hardware; therefore, fixed-point is adopted. Fixed-point representation reduces area and power compared to floating-point implementations [10].

V. RTL Hardware Decoder Architecture

A. Functional Overview

The dfplayer module converts a recognised 11-bit gesture code into a valid UART command frame compatible with the DFPlayer Mini MP3 module. The controller performs three primary operations:

1. Gesture change detection
2. DFPlayer command frame construction
3. UART-based serialised transmission

Additionally, the module integrates real-time latency measurement functionality to evaluate system responsiveness.

B. Gesture Change Detection Mechanism

A register stores the previous gesture state. Whenever:

$$gesture_code \neq prev_gesture$$

A new transmission cycle is initiated.

This prevents redundant frame transmission for stable gesture inputs and ensures:

1. Reduced UART traffic
2. Lower power consumption
3. Deterministic behavior

The detection logic operates synchronously on the positive edge of clk.

C. DFPlayer Command Frame Construction

The DFPlayer Mini uses a 10-byte communication protocol:

Byte Index	Description
0	Start byte
1	Version
2	Length
3	Command
4	Feedback
5	Parameter MSB
6	Parameter LSB
7	Checksum MSB
8	Checksum LSB
9	End byte

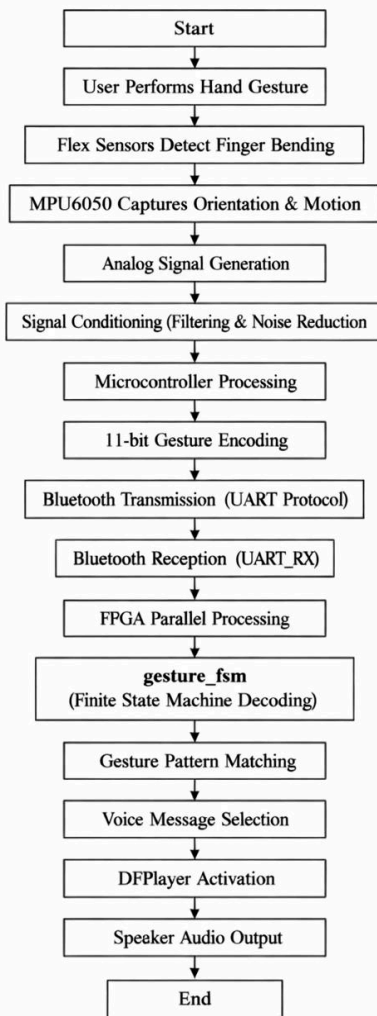
The track number is mapped from the 11-bit gesture using a case statement.

D. Finite State Machine (FSM) Design

The controller responsible for UART-based gesture frame transmission is implemented as a synchronous finite state machine (FSM) to ensure deterministic, structured communication. The FSM is designed to regulate byte-wise data transfer, maintain transmission order, and enable cycle-accurate latency measurement. A Moore architecture is adopted to guarantee stable output behaviour, as outputs depend solely on the current state, thereby eliminating transient glitches during state transitions.

The controller consists of three operational states: **IDLE**, **LOAD**, and **WAIT**. The state sequencing enforces controlled frame buffering, transmission triggering, and completion monitoring.

GestureSpeak: Real-Time Gesture to Voice Conversion Using VLSI



1. IDLE State

The IDLE state represents the default resting condition of the controller. In this state, the system continuously monitors the gesture detection interface for a valid input event. Upon recognition of a new gesture, the following initialisation procedures are performed:

- Resetting the latency counter to ensure accurate timing measurement for the upcoming frame.
- Preparation and loading of the UART frame buffer containing the protocol-compliant command sequence.
- Resetting the internal frame index pointer to the first byte position.

This state guarantees deterministic initialisation before transmission. No UART activity is initiated

until a valid gesture event is detected, thereby minimising unnecessary switching activity and dynamic power consumption.

2. LOAD State

The LOAD state performs controlled data transfer from the internal frame buffer to the UART transmitter interface. In this state:

- The current byte indexed in the frame buffer is transferred to the UART data register.
- A single-cycle send/trigger signal is asserted to initiate transmission.

The LOAD state ensures that only one byte is issued per transmission cycle, preventing bus contention and multiple-trigger hazards. By isolating data loading into a dedicated state, the design enforces strict temporal separation between data setup and transmission monitoring phases.

3. WAIT State

The WAIT state supervises the completion of UART transmission for the currently loaded byte. During this period:

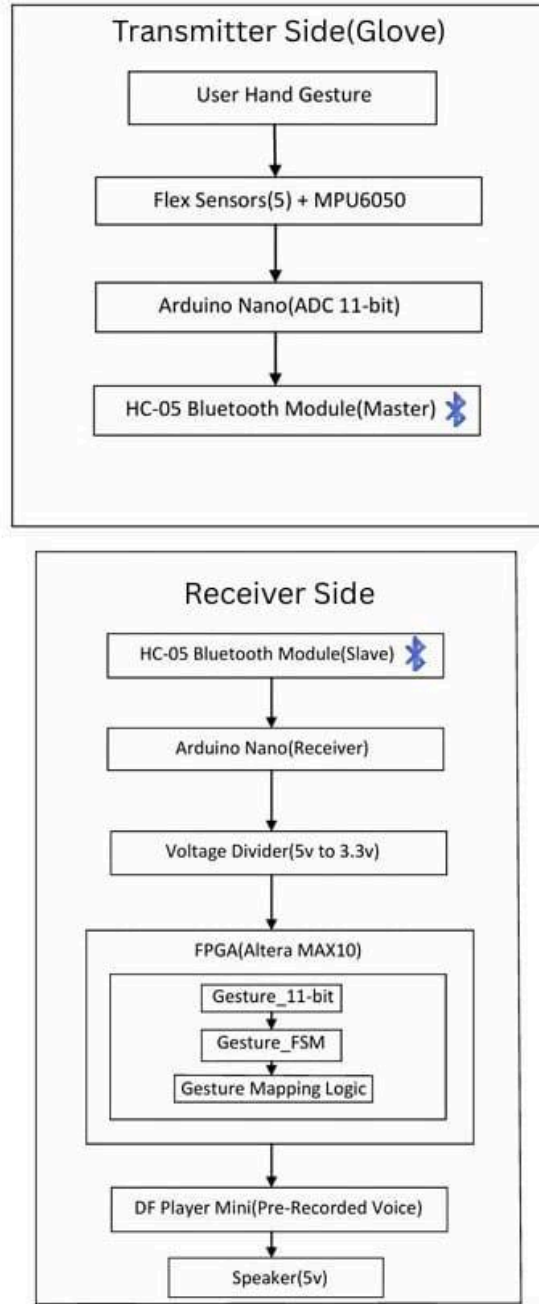
- The controller monitors the transmission-complete flag from the UART module.
- The latency counter increments synchronously with the system clock, enabling cycle-accurate measurement of frame transmission time.

Upon assertion of the transmission-complete signal, the frame index is incremented. If additional bytes remain in the frame buffer, control transitions back to the LOAD state. Otherwise, once all bytes are transmitted, the FSM returns to the IDLE state, concluding the frame transaction. This structured handshake mechanism ensures reliable byte sequencing and prevents premature overwriting of transmission data.

The three-state FSM architecture provides:

- 1) Structured byte-wise frame transmission
- 2) Deterministic state transitions
- 3) Accurate latency accumulation
- 4) Clear separation of initialisation, transmission, and monitoring phases

Since all state transitions are synchronised to the system clock, metastability risks are mitigated, and timing closure is simplified during synthesis. The limited state count reduces combinational complexity, contributing to low resource utilisation and predictable propagation delay.



VI. UART Transmitter Design

A. UART Communication Parameters

The UART transmitter is parameterised as, Clock Frequency = 50 MHz, Baud Rate = 9600 bps

$$\text{Clock cycles per bit} = \frac{50,000,000}{9600} = 5208$$

B. UART Frame Format

Total number of bits=10

Bit Type	Value
Start Bit	0
Data Bit	8 (LSB first)
Stop Bit	1

C. UART FSM States

The UART transmitter is implemented using a synchronous finite state machine (FSM) to ensure protocol-compliant serial data transmission. The architecture follows the standard 8N1 format (8 data bits, no parity, 1 stop bit) and is structured into four primary states: **IDLE**, **START**, **DATA**, and **STOP**. The FSM guarantees precise bit timing, controlled sequencing, and deterministic serial output behaviour

1. **IDLE State** - The IDLE state represents the inactive condition of the UART transmitter.
2. **START State** - The START state initiates serial communication by generating the start bit.
3. **DATA State** - The DATA state performs sequential serialisation of the 8-bit parallel input data

Mathematically, the input data word is represented as, $D = \{d_7, d_6, d_5, d_4, d_3, d_2, d_1, d_0\}$

The transmitted sequence is:

$$d_7 \rightarrow d_6 \rightarrow d_5 \rightarrow d_4 \rightarrow d_3 \rightarrow d_2 \rightarrow d_1 \rightarrow d_0$$

4. **STOP State** - The STOP state finalises the UART frame. The transmission line is driven HIGH for one baud interval.

D. Transmission time analysis

Time per byte:

$$T_{\text{byte}} = \frac{10}{9600} = 1.0417ms$$

Total DFPlayer frame (10 bytes):

$$T_{\text{frame}} = 10.417ms$$



VII. Gesture FSM Design

Each gesture is encoded as a unique 11-bit pattern.

Example:

Gesture	Encoding
COME	01111010000
HELP	11111000100
WATER	01111000100

VIII. Cost Estimation

External hardware components:

S.NO	Components	Quantity	Price (approx.)
1	Flex sensor	10	1000-1200
2	MPU 6050	2	350-400
3	Bluetooth Module	1	300
4	DF Player (Audio module)	1	250
5	Speaker	1	300
6	Battery (Li-ion pack)	1	600
7	Passive components (resistors, capacitors, connectors)	-	800
8	ADC	1	250-300
	Total		~14900

Based on the Cadence simulation results, the proposed design occupies an area of approximately **5013.7 μm^2** , indicating a compact layout suitable for ultra-low-power digital logic applications. The implementation follows a standard-cell ASIC architecture, which allows efficient synthesis, placement, and routing using pre-characterised logic cells. Given the moderate area and the focus on low power consumption rather than extreme performance scaling, such a design would typically be fabricated using a **180 nm or 130 nm CMOS technology node**, which provides a good balance between manufacturing cost, design complexity, and power efficiency for many embedded and low-power digital systems.

A. NRE (Non-Recurring Engineering Cost)

NRE includes design, mask creation, and prototype fabrication before mass production. [10]-[14]

S.NO	Design Stage	Estimated Cost (₹)
1	Specification & architecture design	25,00,000
2	RTL design and verification	55,00,000
3	Physical design (place & route)	60,00,000
4	Mask set generation	70,00,000
5	Prototype tape-out and validation	40,00,000
6	TOTAL NRE	₹2.5 – ₹3.3 Crore(Approx.)

B. Manufacturing cost per chip

S.NO	Manufacturing Component	Estimated Cost (₹) (Approx.)
1	Silicon wafer fabrication	110
2	Packaging	70
3	Testing and QA	45
4	Assembly and logistics	25
	TOTAL COST	~250

C. Manufacturing cost for 50lakh chip

S.NO	Category	Estimated Cost(₹) (Approx.)
1	External hardware per device	14,900
2	ASIC manufacturing cost per chip	250
3	Total chips produced	50,00,000
4	Manufacturing investment	125 Crore
5	NRE one-time design cost	3.3 Crore

IX. Conclusion

The proposed FPGA-based gesture-controlled DFPlayer system is highly suitable for ASIC migration due to its fully synchronous, low-complexity, and synthesizable RTL architecture. The design requires minimal modification, primarily in reset handling, memory optimisation, and clock gating for power efficiency. The estimated silicon area is below 6K gates, and power consumption can be maintained under 2–5 mW depending on process technology. Given its small size and low switching activity, the design is ideal for low-cost, battery-powered assistive communication devices. While ASIC development introduces higher upfront costs and reduced flexibility compared to FPGA implementation, it significantly improves power efficiency, miniaturization, and per-unit cost in high-volume production scenarios. Therefore, the system demonstrates strong feasibility for commercial ASIC realisation in healthcare and wearable communication applications.

X. References

- [1] S. Mitra and T. Acharya, "Gesture recognition: A survey," *IEEE Trans. Syst., Man, Cybern.*, vol. 37, no. 3, 2007.
- [2] R. Lockton and A. Fitzgibbon, "Real-time gesture recognition," *Proc. IEEE CVPR*, 2002.
- [3] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, *Digital Integrated Circuits*, 2nd ed., Prentice Hall.
- [4] D. Feng et al., "Design and implementation of a gesture recognition system based on flex sensors," *IARJSET*, 2024.
- [5] S. Sharma et al., "Quantized neural networks on FPGA," *IEEE Access*, vol. 12, 2024.
- [6] Y. Guo et al., "FPGA-based QDS-CNN system for sEMG gesture and force recognition," *IEEE Trans. Biomed. Circuits Syst.*, 2024.
- [7] M. Jaiswal et al., "Quantized CNN-based hardware for real-time hand gesture recognition," *Microelectronics Journal*, 2024.
- [8] R. Duarte et al., "Design of real-time gesture recognition with CNN on low-end FPGA," *Electronics*, 2025.
- [9] FPGA real-time hand gesture recognition using lightweight CNN on PYNQ-Z2, *IJISRT*, 2026.
- [10] **Mouser Electronics**, "Electronic component distributor catalog," Mouser Electronics Inc. [Online]. Available: <https://www.mouser.in>. Accessed: Mar. 2026.
- [11] **DigiKey Electronics**, "Electronic components and semiconductor distributor," DigiKey Electronics. [Online]. Available: <https://www.digikey.in>. Accessed: Mar. 2026.
- [12] **Texas Instruments**, "Analog integrated circuits product catalog," Texas Instruments Inc. [Online]. Available: <https://www.ti.com>. Accessed: Mar. 2026.
- [13] **Silicon Analysts**, *Foundry Engagement Guide – Wafer and Mask Cost Estimates*. Silicon Analysts Industry Report.
- [14] **Electronic Design**, "The economics of ASIC design and manufacturing," *Electronic Design Magazine*. [Online].